```
# Install required libraries for Transformer models and dataset handling
# transformers → pretrained models like BERT / DistilBERT
# datasets → provides ready-to-use datasets like IMDB
# torch → deep learning framework used by Hugging Face models
# scikit-learn → used for evaluation metrics
# pandas → data manipulation

!pip install transformers datasets torch scikit-learn pandas
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transfo
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from tran
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from tran
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from trans
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from trans
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from to
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from sci
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from sc
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pan
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from p
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingfac
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from req
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from ai
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from ht
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpco
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typ
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typ
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markd
```

```
# Import pandas for data handling
import pandas as pd
```

```python
# Import torch which is the deep learning framework used for training the model
import torch

# Import load_dataset to download datasets directly from Hugging Face
from datasets import load_dataset

# Import tokenizer and pretrained model for sequence classification
from transformers import (
    DistilBertTokenizerFast,          # Converts text into tokens understandable by the model
    DistilBertForSequenceClassification, # Pretrained DistilBERT model for classification tasks
    Trainer,                          # Simplifies model training and evaluation
    TrainingArguments                 # Contains configuration parameters for training
)

# Import evaluation metrics from sklearn
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
```

```python
# Load the IMDB movie reviews dataset from Hugging Face
# This dataset contains movie reviews labeled as positive or negative sentiment

dataset = load_dataset("imdb")

# Print dataset information such as number of samples and features
print(dataset)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingfac
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datas
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable h
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hu
```

README.md: 7.81k/? [00:00<00:00, 517kB/s]

plain_text/train-00000-of-00001.parquet: 100% 21.0M/21.0M [00:00<00:00, 105MB/s]

plain_text/test-00000-of-00001.parquet: 100% 20.5M/20.5M [00:00<00:00, 102MB/s]

plain_text/unsupervised-00000-of-00001.p(…): 100% 42.0M/42.0M [00:00<00:00, 67.1MB/s]

Generating train split: 100% 25000/25000 [00:00<00:00, 82482.79 examples/s]

Generating test split: 100% 25000/25000 [00:00<00:00, 86247.86 examples/s]

Generating unsupervised split: 100% 50000/50000 [00:00<00:00, 116620.64 examples/s]

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label'],
        num_rows: 50000
    })
})
```

```
# Display the first sample from the training dataset
# Each sample contains:
# text → movie review
# label → sentiment (0 = negative, 1 = positive)

print(dataset['train'][0])
```

```
{'text': 'I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that sur
```

```
# Load the pretrained tokenizer corresponding to DistilBERT
# Tokenizer converts raw text into numerical tokens that the model understands

tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-uncased")

# Print confirmation message
print("Tokenizer loaded successfully")
```

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 4.14kB/s]

vocab.txt: 232k/? [00:00<00:00, 6.22MB/s]

tokenizer.json: 466k/? [00:00<00:00, 6.73MB/s]

```
Tokenizer loaded successfully
```

```
# Define a function to tokenize the text reviews
# Tokenization converts sentences into token IDs required by the model

def tokenize_function(example):

    # Tokenize the review text
    # truncation=True → cuts long reviews exceeding max length
    # padding="max_length" → ensures all sequences are the same length
    # max_length=256 → limits sequence size to reduce memory usage
    return tokenizer(
        example["text"],
        truncation=True,
        padding="max_length",
        max_length=256
    )

# Apply tokenization to the entire dataset
# batched=True → processes multiple samples together for faster execution
tokenized_datasets = dataset.map(tokenize_function, batched=True)

# Print confirmation message
print("Tokenization completed successfully")
```

Map: 100% 25000/25000 [00:27<00:00, 792.33 examples/s]

Map: 100% 25000/25000 [00:28<00:00, 842.16 examples/s]

Map: 100% 50000/50000 [00:56<00:00, 970.52 examples/s]

```
Tokenization completed successfully
```

```
# Remove the original text column because the model only needs tokenized inputs
tokenized_datasets = tokenized_datasets.remove_columns(["text"])

# Rename "label" column to "labels" because Hugging Face Trainer expects this name
tokenized_datasets = tokenized_datasets.rename_column("label", "labels")

# Convert dataset into PyTorch tensors
tokenized_datasets.set_format("torch")

# Print dataset structure after formatting
print(tokenized_datasets)
```

```
DatasetDict({
    train: Dataset({
        features: ['labels', 'input_ids', 'attention_mask'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['labels', 'input_ids', 'attention_mask'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['labels', 'input_ids', 'attention_mask'],
        num_rows: 50000
    })
})
```

```
# Extract training dataset
train_dataset = tokenized_datasets["train"]

# Extract testing dataset
test_dataset = tokenized_datasets["test"]

# Print the number of training and testing samples
print("Training samples:", len(train_dataset))
print("Testing samples:", len(test_dataset))
```

```
Training samples: 25000
Testing samples: 25000
```

```
# Load the DistilBERT pretrained model for sequence classification
# num_labels=2 indicates binary sentiment classification (positive / negative)

model = DistilBertForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=2
)

# Print confirmation message
print("Pretrained DistilBERT model loaded successfully")
```

```
config.json: 100%                                    483/483 [00:00<00:00, 20.2kB/s]

model.safetensors: 100%                              268M/268M [00:05<00:00, 78.1MB/s]

Loading weights: 100%  100/100 [00:00<00:00, 364.10it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.w

DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                     | Status     |
------------------------+------------+-
vocab_transform.weight  | UNEXPECTED |
vocab_transform.bias    | UNEXPECTED |
vocab_projector.bias    | UNEXPECTED |
vocab_layer_norm.bias   | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
pre_classifier.bias     | MISSING    |
classifier.bias         | MISSING    |
pre_classifier.weight   | MISSING    |
classifier.weight       | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expec
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider tra
Pretrained DistilBERT model loaded successfully
```

```
# Define a function that calculates evaluation metrics during model evaluation
# Metrics include accuracy, precision, recall, and F1-score

def compute_metrics(eval_pred):

    # Extract predicted logits and true labels
    logits, labels = eval_pred

    # Convert logits into predicted class labels
    predictions = logits.argmax(axis=1)

    # Calculate precision, recall, and F1-score
    precision, recall, f1, _ = precision_recall_fscore_support(
        labels,
        predictions,
        average='binary'
    )

    # Calculate accuracy
    acc = accuracy_score(labels, predictions)

    # Return metrics as a dictionary
    return {
        "accuracy": acc,
        "f1": f1,
        "precision": precision,
        "recall": recall
    }
```

```
training_args = TrainingArguments(

    # Directory where training results will be stored
    output_dir="./results",

    # Evaluate the model after each epoch
    eval_strategy="epoch",

    # Learning rate for optimizer
    learning_rate=2e-5,

    # Batch size for training
    per_device_train_batch_size=8,
```

```
        # Batch size for evaluation
        per_device_eval_batch_size=8,

        # Number of training epochs
        num_train_epochs=2,

        # Weight decay for regularization
        weight_decay=0.01,

        # Directory to store logs
        logging_dir="./logs"
    )

    # Print confirmation
    print("Training arguments configured")
```

```
`logging_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD_LOGGING_DIR` inst
Training arguments configured
```

```
# Initialize Trainer which handles training, evaluation, and prediction

trainer = Trainer(

    # Model to be trained
    model=model,

    # Training configurations
    args=training_args,

    # Training dataset
    train_dataset=train_dataset,

    # Evaluation dataset
    eval_dataset=test_dataset,

    # Function used to compute evaluation metrics
    compute_metrics=compute_metrics
)

# Print confirmation
print("Trainer initialized successfully")
```

```
Trainer initialized successfully
```

```
# Start model training
# Trainer automatically handles forward pass, backpropagation, and optimization

trainer.train()

# After training completes, loss should decrease indicating learning
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memc
  super().__init__(loader)
```

[191/6250 33:58 < 18:09:20, 0.09 it/s, Epoch 0.06/2]

Epoch	Training Loss	Validation Loss

[369/6250 1:05:14 < 17:25:34, 0.09 it/s, Epoch 0.12/2]

Epoch	Training Loss	Validation Loss

```
# Evaluate the trained model on the test dataset

results = trainer.evaluate()
```

```
# Print evaluation metrics such as accuracy, precision, recall, and F1-score
print("Evaluation Results:")
print(results)
```

```
# Provide a custom sentence for sentiment prediction
text = "The movie was absolutely fantastic with brilliant acting!"

# Tokenize the input sentence
inputs = tokenizer(
    text,
    return_tensors="pt",  # return PyTorch tensors
    truncation=True,
    padding=True
)

# Pass inputs through the trained model
outputs = model(**inputs)
```